Vídeo 6.4: Usando a API com responsabilidade

Duração-alvo: 7 min | **Conceitos:** dados pessoais, configurações e segredos, .gitignore e limites da API | **O aluno sai sabendo:** o que não pode ir para o repositório, como separar as configurações do código e o que ainda falta antes de uma API ir para a internet.

Antes de gravar: partir do código do vídeo 6.3. Conferir que a versão do Node.js aceita --env-file (Node 20.6 ou mais recente). Ao fim da gravação, marcar o código como versão v6.

[0:00] Abertura

Na tela: título do vídeo.

Fala: “Chegamos ao último vídeo do curso. A sua API funciona, valida, tem testes e documentação. Antes de compartilhá-la com o mundo, falta um assunto que separa quem só programa de quem programa com responsabilidade: cuidar do que não deve ficar exposto, e cuidar das pessoas por trás dos dados.”

[0:50] Dados são de pessoas

Na tela: três hábitos em uma lista: coletar só o necessário, avisar para que serve, proteger o que guarda.

Fala: “A nossa biblioteca guarda só título e autor, que são dados públicos. Mas, se um dia você guardar nomes, e-mails ou telefones, estará lidando com dados pessoais, e no Brasil existe uma lei para isso, a LGPD, a Lei Geral de Proteção de Dados. Três hábitos simples: colete só o que realmente precisa; avise para que vai usar; e proteja o que guarda. E, nos exemplos e nos testes, use sempre dados fictícios, como fizemos aqui. Nunca use dados de pessoas reais para testar.”

[2:00] Configurações e segredos fora do código

Na tela: criar, na raiz do projeto, os três arquivos abaixo, um de cada vez.

```
# .env
 PORT=3000
 ARQUIVO_BANCO=biblioteca.db


# .gitignore
 node_modules
 .env
 *.db


# .env.example
 PORT=
 ARQUIVO_BANCO=
```

Fala: “Configurações como porta, senhas e chaves de serviços não devem ficar escritas no código, e muito menos no repositório público do GitHub. O hábito é colocá-las em um arquivo chamado ponto env: uma linha para cada configuração, com o nome, um sinal de igual e o valor. Hoje temos duas, a porta e o arquivo do banco, e amanhã poderia ter uma

senha. Depois criamos o ponto gitignore, que é a lista do que o Git deve ignorar e nunca enviar para o GitHub. Colocamos nele a pasta node_modules, que é pesada e se refaz com npm install; o ponto env, que guarda os segredos; e os arquivos terminados em ponto db, que são o banco de dados com os dados reais. E, para quem for usar o projeto, deixamos o ponto env ponto example, só com os nomes das configurações, sem os valores: é o roteiro do que precisa ser preenchido."

[3:45] Lendo as configurações no código

Na tela: alterar o src/server.js, depois rodar o servidor com o comando abaixo. Mudar a porta no .env para 4000 e rodar de novo.

```
const app = require('./app');

 const porta = process.env.PORT || 3000;

 app.listen(porta, () => {
   console.log(`Servidor rodando em http://localhost:${porta}`);
 });
```

```
node --env-file=.env src/server.js
```

Fala: "No server ponto js, trocamos o número fixo por process ponto env ponto PORT. Se existir a configuração PORT, o programa usa; senão, usa 3000. Para o Node ler o arquivo ponto env, acrescentamos traço traço env-file igual a ponto env ao comando: node, traço traço env-file igual ponto env, src barra server ponto js. Vou mudar a porta no ponto env para 4000 e rodar: o servidor sobe na 4000. A mudança foi só no arquivo de configuração, não no código."

[5:00] A lista antes de publicar

Na tela: a lista abaixo, aparecendo um item de cada vez.

- Dados fictícios nos exemplos e nos testes.
- Nenhum segredo no código nem no repositório.
- Erros que não revelam detalhes internos.
- Testes passando (npm test).
- Documentação atualizada (/docs).
- Licença no repositório.

Fala: "Antes de publicar uma API, passe por esta lista. Dados fictícios. Nenhum segredo no código nem no repositório. Mensagens de erro que não entregam detalhes técnicos, e isso já fizemos. Testes passando. Documentação atualizada. E uma licença no repositório, dizendo o que as outras pessoas podem fazer com o seu trabalho. Neste projeto, os textos usam a licença CC BY 4.0, e o código usa a licença MIT."

[5:45] O que ainda falta

Na tela: duas palavras, "autenticação" e "autorização", com uma frase curta ao lado de cada uma.

Fala: "Uma honestidade importante: a nossa API não tem login. Qualquer pessoa que consiga acessá-la pode cadastrar e excluir livros. Para estudar no seu computador, tudo

bem. Antes de colocar uma API na internet com dados reais, é preciso adicionar autenticação, que é verificar quem está pedindo, e autorização, que é verificar o que cada pessoa pode fazer.”

[6:20] Próximos passos e despedida

Na tela: uma lista de caminhos: autores e empréstimos, login, hospedagem, uma tela que use a API. No fim, o endereço do repositório e a licença.

Fala: “E agora? Você construiu, do zero, uma API completa. Alguns próximos passos: acrescentar autores e empréstimos à biblioteca, incluir login, publicar a API em um serviço de hospedagem ou criar uma tela que use a API. Um lembrete: o GitHub Pages, onde este curso está, só mostra páginas. Para publicar uma API feita em Node ponto js, é preciso um serviço que execute o programa. E, se você encontrou algum erro ou barreira neste curso, abra uma Issue no repositório: o curso é aberto e melhora com o seu retorno. Obrigado por ter vindo até aqui!”

Exercício sugerido: revisar o próprio projeto com a lista antes de publicar e anotar o que falta atender.